

Unit Test Extension Report

Course: Software Quality — SQ_2026i

Team: F

System: [saleor/saleor-platform](https://github.com/saleor/saleor-platform)

Functional flow: Product Listing — Dashboard → API resolver → Database

Isolation in unit testing provides **fast, deterministic, and diagnosable** verification of each tier. In our implementation, frontend tests do not contact the real API, backend tests do not contact the real database, and database tests run directly on an isolated PostgreSQL schema instead of through application code. The architectural benefit is that failures are localized: a regression in SQL constraints fails only the DB suite, a resolver defect fails only backend tests, and a UI or API-consumption issue fails only frontend tests. What would be lost without this isolation is precision in debugging, execution speed, and confidence in automation; tests would become flaky cross-tier integration checks instead of reliable unit tests.

- **Database tier:** the `product_product` verification scripts (`INSERT`, `SELECT`, `UPDATE`, `DELETE`, `CONSTRAINTS`) validate SQL behavior directly in an isolated schema created by `init_test_schema.sql` and executed by `run-tests.sh`. Here the database is the unit under test, so isolation is achieved with a dedicated schema and controlled seed data rather than mocks. This follows the laboratory principle that DB unit tests must validate the DB tier itself, independently from backend code.
- Backend tier: `resolve_products()` is tested with a Stub (`mock_info`) and a Mock (`mock_info_spy`), supported by an in-memory fake query/manager behavior. The stub provides only the resolver context needed for execution, while the mock verifies delegation such as `.all()` and `.filter(...)`. This keeps the test independent from Django request lifecycle and PostgreSQL, applying both state-based testing and interaction-based testing.
- Frontend tier: `fetchProducts()` and the `ProductList` boundary are tested with a Stub on `global.fetch` and a Mock on `jest.mock("../api")`. The fetch stub allows deterministic verification of GraphQL request/response handling without contacting the backend, while the module mock isolates the React component so the DOM remains the unit under test. This follows the lab principle that frontend unit tests must not contact the real backend and should mock API calls explicitly.

Our automated execution confirms this design: DB, backend, and frontend suites run independently in Docker and exit with code 0, which demonstrates correct tier isolation and reproducible unit-level verification.